Decomposition

Decomposition means breaking problems down into sub-problems in order to explain a process more clearly. Decomposition leads us to the concept of program modules and using procedures and functions.

Data modelling

Data modelling involves analysing and organising data (see Chapter 13). We can set up abstract data types to model real-world concepts, such as queues or stacks. When a programming language does not have such data types built-in, we can define our own by building them from existing data types. There are more ways to build data models. In Chapter 27 we cover object-oriented programming where we build data models by defining classes. In Chapter 29 we model data using facts and rules. In Chapter 26 we cover random files.

Pattern recognition

Pattern recognition means looking for patterns or common solutions to common problems and using these to complete tasks in a more efficient and effective way. There are many standard algorithms to solve standard problems, such as sorting (see Section 13.03 and 23.03) or searching (see Section 13.02 and 23.04).

Algorithm design

Algorithm design involves developing step-by-step instructions to solve a problem.

KEY TERM

Algorithm: a sequence of defined steps that can be carried out to perform a task

12.02 What is an algorithm?

We use algorithms in everyday life. If you need to change a wheel on a car, you might need to follow instructions (the algorithm) from a manual.

1. Take a spanner and loosen the wheel nuts.
2. Position a jack in an appropriate place.
3. Raise the car.
4. Take off the wheel nuts and the wheel.
5. Lift replacement wheel into position.
6. Replace wheel nuts and tighten by hand.
7. Lower the car.
8. Fully tighten wheel nuts.

This might sound all very straightforward. However, if the instructions are not followed in the correct logical sequence, the process might become much more difficult or even impossible. For example, if you tried to do Step 1 after Step 3, the wheel may spin and you can't loosen the wheel nuts. You can't do Step 4 before Step 3.

If you want to bake a cake, you follow a recipe.

1. Measure the following ingredients: 200g sugar, 200g butter, 4 eggs, 200g flour, 2 teaspoons baking powder and 2 tablespoons of milk.
2. Mix the ingredients together in a large bowl, until the consistency of the mixture is smooth.

3 Pour the mixture into a cake tin.

4 Bake in the oven at 190° C for 20 minutes.

5 Check it is fully cooked.

6 Turn cake out of the tin and cool on a wire rack.

The recipe is an algorithm. The ingredients are the input and the cake is the output. The process is mixing the ingredients and cooking the mixture in the oven.

Sometimes a step might need breaking down into smaller steps. For example, Step 2 can be more detailed.

2.1 Beat the sugar and butter together until fluffy.

2.2 Add the eggs, one at a time, mixing constantly.

2.3 Sieve the flour and baking powder and stir slowly into the egg mixture.

2.4 Add milk and mix to give a creamy consistency.

Sometimes there might be different steps depending on some other conditions. For example, consider how to get from one place to another using the map of the London Underground system in Figure 12.02.



Figure 12.02 Underground map of London, UK

To travel from King's Cross St. Pancras to Westminster, we consider two routes:

- Route A: Take the Victoria Line to Green Park (4 stations); then take the Jubilee Line to Westminster (1 station)
- Route B: Take the Piccadilly Line to Green Park (6 stations); then take the Jubilee Line to Westminster (1 station).

Route A looks like the best route. If there are engineering works on the Victoria Line and trains are delayed, Route B might turn out to be the quicker route.

The directions on how to get from King's Cross St. Pancras to Westminster can be written as:

IF there are engineering works on the Victoria Line

THEN

Take the Piccadilly Line to Green Park (6 stations)

Take the Jubilee Line to Westminster (1 station)

ELSE

Take the Victoria Line to Green Park (4 stations)

Take the Jubilee Line to Westminster (1 station)

TASK 12.02

Write the steps to be followed to:

- make a sandwich
- walk from your school/college to the nearest shop
- log on to your computer.

Many problems have more than one solution. Sometimes it is a personal preference which solution to choose. Sometimes one solution will be measurably better than another.

12.03 Expressing algorithms

TIP

Computer scientists are interested in finding good solutions. A good solution gives the correct results, takes up as little computer memory as possible and executes as fast as possible. The solution should be concise, elegant and easy to understand.

In computer science, when we design a solution to a problem we express the solution (the algorithm) using sequences of steps written in **structured English** or **pseudocode**. Structured English is a subset of the English language and consists of command statements. Pseudocode resembles a programming language without following the syntax of a particular programming language. A **flowchart** is an alternative method of representing an algorithm. A flowchart consists of specific shapes, linked together.

KEY TERMS

Structured English: a subset of the English language that consists of command statements used to describe an algorithm

Pseudocode: a way of using keywords and identifiers to describe an algorithm without following the syntax of a particular programming language

Flowchart: shapes linked together to represent the sequential steps of an algorithm

An algorithm consists of a sequence of steps. Under certain conditions we may wish not to perform some steps. We may wish to repeat a number of steps. In computer science, when writing algorithms, we use four basic types of construct.

- **Assignment:** a value is given a name (identifier) or the value associated with a given identifier is changed.
- **Sequence:** a number of steps are performed, one after the other.
- **Selection:** under certain conditions some steps are performed, otherwise different (or no) steps are performed.
- **Repetition:** a sequence of steps is performed a number of times. This is also known as iteration or looping.

Many problems we try to solve with a computer involve data. The solution involves inputting data to the computer, processing the data and outputting results (as shown in Figure 12.03).

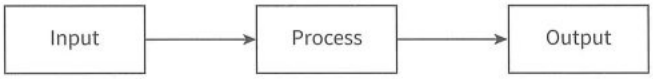
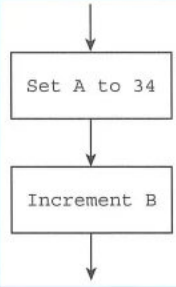


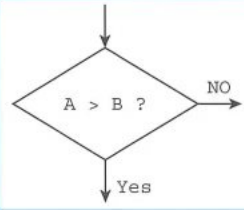
Figure 12.03 Input–process–output

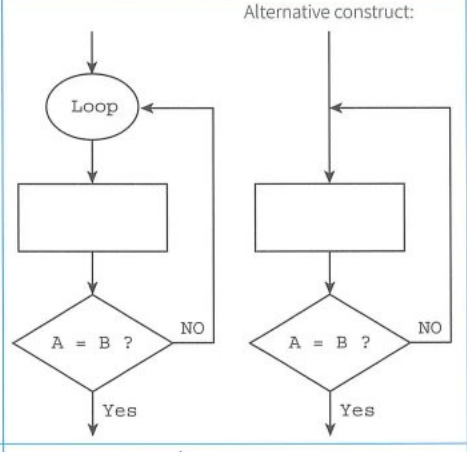
We therefore also need input and output statements.

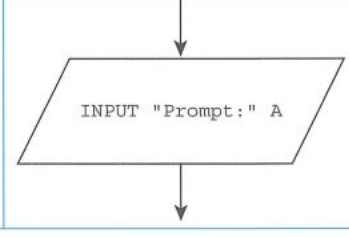
We need to know the constructs so we know how detailed our design has to be. These constructs are represented in each of the three notations as shown in Table 12.01.

In this book, algorithms and program code are typed using the `Courier` font.

	Structured English	Pseudocode	Flowchart
Assignment and Sequence	`SET A TO 34` `INCREMENT B`	`A ← 34` `B ← B + 1`	

Selection	`IF A IS GREATER THAN B`
	`  THEN ...`
	`  ELSE ...`
		`IF A > B`
		`  THEN ...`
		`  ELSE ...`
		`ENDIF`
			

Repetition	`REPEAT UNTIL A IS`
	`EQUAL TO B  ...`
		`REPEAT`
		`  ...`
		`UNTIL A = B`
			

Input	`INPUT A`	`INPUT "Prompt: "` `A`	

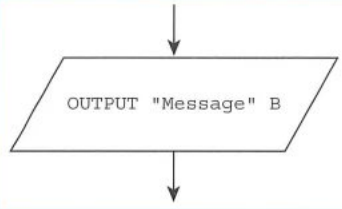
	Structured English	Pseudocode	Flowchart
Output	`OUTPUT "Message"` `OUTPUT B`	`OUTPUT "Message", B`	

Table 12.01 Constructs for computing algorithms

12.04 Variables

When we input data for a process, individual values need to be stored in memory. We need to be able to refer to a specific memory location so that we can write statements of what to do with the value stored there. We refer to these named memory locations as **variables**. You can imagine these variables like boxes with name labels on them. When a value is input, it is stored in the box with the specified name (identifier) on it.

KEY TERM

Variable: a storage location for a data value that has an identifier

For example, the variable used to store a count of how many guesses have been made in a number guessing game might be given the identifier `NumberOfGuesses` and the player's name might be stored in a variable called `ThisPlayer`, as shown in Figure 12.04.

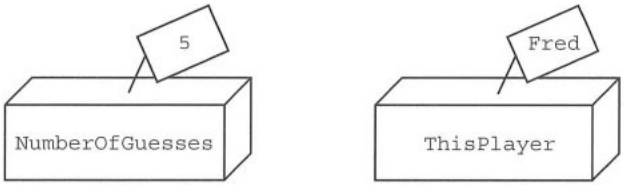


Figure 12.04 Variables

Variable identifiers should not contain spaces, only letters, digits and _ (the underscore symbol). To make algorithms easier to understand, the naming of a variable should reflect the variable's use. This means often that more than one word is used as an identifier. The formatting convention used here is known as CamelCaps. It makes an identifier easier to read.

12.05 Assignments

Assigning a value

The following pseudocode stores the value that is input (for example 15) in a variable with the identifier `Number` (see Figure 12.05(a)).

```
INPUT Number
```

The following pseudocode stores the value 1 in the variable with the identifier `NumberOfGuesses` (see Figure 12.05(b)).

```
NumberOfGuesses ← 1
```